

Núcleo de conocimiento: asistente de WhatsApp con n8n

Base técnica para construir, sin errores, un asistente de WhatsApp usando la API oficial de Meta (Cloud API) orquestada desde n8n. Investigado sobre documentación oficial de Meta y n8n (2026). El objetivo de este archivo es que quien lo use pueda explicar cada paso, no solo ejecutarlo.

1. Arquitectura general

Cliente escribe por WhatsApp

- Meta WhatsApp Cloud API recibe el mensaje
- Envía un webhook a n8n (nodo WhatsApp Trigger)
- n8n procesa (reglas de palabras clave, o nodo de IA, o ambos)
- n8n consulta la fuente de datos (Google Sheet)
- n8n responde vía el nodo WhatsApp Business Cloud
- El cliente recibe la respuesta en su WhatsApp normal

Meta **hospeda** la infraestructura de mensajería (por eso se llama "Cloud API" — a diferencia de la versión "On-Premises" vieja, ya discontinuada). n8n actúa como el "cerebro" que decide qué responder.

2. Cuenta y credenciales — cómo evitar el error más común

- Se necesita una **cuenta de Meta Business** verificada, una app de tipo Business en developers.facebook.com, con el producto "WhatsApp" agregado.
- **Error común a evitar:** usar un token de usuario personal. Estos expiran cada 60 días y rompen el bot sin aviso. La práctica correcta es crear un **System User** (Configuración del negocio → Usuarios del sistema) con permisos `whatsapp_business_messaging` y `whatsapp_business_management`, y generar un token desde ahí — puede configurarse para que **no expire**, evitando el mantenimiento recurrente.
- El token se pasa como Bearer token en el header `Authorization` de cada request (esto lo maneja n8n automáticamente si se configura como credencial, en vez de hardcodearlo en cada nodo).

3. Los dos nodos clave en n8n

- **WhatsApp Trigger** (a veces llamado "On Messages"): dispara el flujo cuando llega un mensaje nuevo o cambia el estado de uno enviado. Es el punto de entrada.
- **WhatsApp Business Cloud** (nodo de acción): envía mensajes. Soporta "Send Message" (texto libre), "Send Template" (plantillas aprobadas), y manejo de multimedia (subir, bajar, borrar). Si alguna operación puntual no está soportada por este nodo, se puede usar el nodo genérico **HTTP Request** apuntando directo a la Graph API de Meta, reusando la misma credencial.

4. Estructura real del webhook (para no romper el parseo)

Cada webhook de Meta llega con esta forma (simplificada):

```
{
  "object": "whatsapp_business_account",
  "entry": [{
    "id": "...",
    "changes": [{
      "value": {
        "messaging_product": "whatsapp",
        "metadata": { "display_phone_number": "...", "phone_number_id": "..." },
        "contacts": [{ "profile": { "name": "..." }, "wa_id": "..." }],
        "messages": [{
          "from": "...",
          "id": "wamid...",
          "timestamp": "...",
          "text": { "body": "..." },
          "type": "text"
        }]
      },
      "field": "messages"
    }]
  }]
}
```

Puntos clave que hay que manejar bien:

- El texto del mensaje del cliente está en `entry[0].changes[0].value.messages[0].text.body` — no en la raíz del payload. Un parseo mal armado que busque el texto en el lugar equivocado es el bug más común al empezar.

- El campo `type` puede ser `text`, `image`, `document`, `location`, `interactive`, entre otros — el flujo debe filtrar o manejar cada tipo, no asumir que todo es texto.
- Para actualizaciones de estado (entregado, leído), la información viene en `statuses[]` en vez de `messages[]` — son eventos distintos que llegan al mismo webhook.

5. Reglas no negociables del webhook

- **Responder HTTP 200 dentro de los 5 segundos.** Si el procesamiento (consultar el Sheet, llamar a una IA) tarda más, hay que responder 200 primero y procesar de forma asíncrona — si no, Meta lo interpreta como fallo.
- **Meta reintenta durante hasta 7 días** si no recibe un 200, con frecuencia decreciente. Esto significa que **pueden llegar mensajes duplicados** — el flujo tiene que ser capaz de ignorar un mensaje que ya procesó (usar el `id` del mensaje, tipo `wamid...`, como identificador único para no responder dos veces lo mismo).
- **Verificar la firma del webhook** (viene en el header `X-Hub-Signature-256`) antes de procesar cualquier payload — sin esto, cualquiera que descubra la URL del webhook podría mandarle datos falsos al flujo.
- El payload puede pesar hasta 3 MB — relevante si en algún momento se procesan imágenes/audios.

6. La ventana de servicio de 24 horas — el concepto más importante de todos

Esta es la regla que más errores genera si no se entiende bien:

- Cuando un cliente te escribe, se abre una **ventana de 24 horas** durante la cual podés responder con **texto libre**, sin restricciones de formato.
- **Pasadas esas 24 horas sin que el cliente vuelva a escribir, ya NO se puede mandar texto libre.** Solo se puede iniciar contacto de nuevo usando un **mensaje de plantilla (template)** pre-aprobado por Meta.
- Las plantillas tienen 3 categorías: **Marketing**, **Utility** (confirmaciones, recordatorios ligados a una acción del cliente) y **Authentication** (códigos de un solo uso, con reglas de contenido más estrictas, sin texto de marketing).
- Las plantillas se crean en Meta Business Suite o vía API, y Meta las revisa — la aprobación toma típicamente hasta 24 horas. No se pueden usar el mismo día que se crean.

- **Para el caso de Librería Neneko:** como el bot solo responde consultas entrantes (nunca inicia contacto), esto no debería ser un problema en la práctica — el cliente escribe, se abre la ventana, el bot responde libremente. Las plantillas solo entrarían en juego si en el futuro quisieran mandar promociones o recordatorios de forma proactiva.

7. Costos (repaso, con el cambio que ya viene)

- Hoy: crear la app, el número, usar n8n y el Google Sheet no tiene costo. Los mensajes dentro de la ventana de 24 horas son gratis.
- **A partir del 1 de octubre de 2026**, Meta empieza a cobrar también estos mensajes de servicio (ver el archivo de costos que armamos antes) — fracciones de centavo a centavos por mensaje según el país, vale la pena confirmar la tarifa exacta de Argentina en la tabla oficial de Meta más cerca de la fecha.

8. Checklist antes de publicar el flujo

- Token del System User configurado (no un token personal de 60 días)
- Verificación de firma del webhook activa
- Manejo de duplicados usando el `id ( wamid... )` del mensaje
- Respuesta HTTP 200 en menos de 5 segundos (procesamiento pesado desacoplado si hace falta)
- Filtro por `type` de mensaje (no asumir que todo es texto)
- Rama de “no matchea ninguna regla” que derive a la dueña, no que falle en silencio
- Número de teléfono dedicado, confirmado que no se usa en paralelo desde la app normal
- Probado en modo test antes de activar con clientes reales

9. Dónde profundizar si hace falta

- Documentación oficial de webhooks de Meta: [developers.facebook.com](https://developers.facebook.com/docs/whatsapp/business-platform/webhooks) → WhatsApp Business Platform → Webhooks
- Documentación de plantillas: [developers.facebook.com](https://developers.facebook.com/docs/whatsapp/business-platform/message-templates) → WhatsApp Business Platform → Message Templates
- Nodo de n8n: docs.n8n.io → Integrations → WhatsApp Business Cloud

